

Autonomous Bioinformatics Agent

Overview

The Autonomous Bioinformatics Agent is an AI-powered system that automates genomic data processing pipelines from start to finish.

Instead of requiring researchers to manually execute bioinformatics tools, debug pipeline failures, and manage infrastructure, the agent acts as an autonomous bioinformatics engineer capable of planning, executing, monitoring, and repairing genomic workflows.

The system transforms a simple user request into a complete variant-calling workflow while automatically handling common computational failures.

Problem Statement

Modern bioinformatics workflows are complex and fragile.

Researchers often spend significant time:

- Writing workflow scripts
- Managing dependencies
- Running command-line tools
- Diagnosing pipeline failures
- Fixing resource and configuration issues

Current workflow managers automate execution but still require human intervention when pipelines fail.

Our goal is to build an AI agent that can execute workflows and recover from failures autonomously.

Vision

Create an AI Bioinformatics Engineer that can:

1. Understand analysis requests.
2. Build genomic workflows automatically.
3. Execute pipelines.
4. Monitor logs in real time.
5. Diagnose failures.
6. Apply corrective actions.
7. Resume execution.

8. Deliver final results and reports.

Core User Flow

User Input:

Analyze SRR12345678 against hg38 and generate variants.

Agent Workflow:

SRA ID ↓ Download Reads ↓ Quality Control ↓ Alignment ↓ Sorting & Indexing ↓ Variant Calling ↓
Report Generation ↓ Database Upload

Final Output:

- BAM file
 - VCF file
 - QC report
 - Analysis summary
 - Execution log
-

Key Features

1. Natural Language Interface

Users can submit requests such as:

- Analyze SRR12345678
- Call variants against hg38
- Generate a genomic report

No workflow scripting required.

2. Automated Pipeline Generation

The agent dynamically constructs workflows based on user requirements.

Example tools:

- BWA-MEM2
- samtools

- bcftools
 - FastQC
 - MultiQC
-

3. Autonomous Execution

The system executes tools through Python subprocess management.

Capabilities:

- Command execution
 - Resource monitoring
 - Progress tracking
 - Output capture
-

4. Real-Time Log Monitoring

The agent continuously observes:

- stdout
- stderr
- exit codes

to identify failures immediately.

5. Failure Diagnosis

The AI analyzes logs and classifies issues such as:

- Missing reference indexes
 - File path errors
 - Out-of-memory failures
 - Missing dependencies
 - Disk space issues
 - Invalid parameters
-

6. Self-Healing Workflows

The system automatically applies corrective actions.

Examples:

Missing BWA Index

Detected:

Reference index not found

Action:

Generate BWA index

Result:

Pipeline resumes automatically

Out-of-Memory

Detected:

Process terminated due to memory limits

Action:

Reduce thread count

Result:

Pipeline continues successfully

7. Report Generation

After execution, the system produces:

- Coverage metrics
 - Variant statistics
 - QC summaries
 - Execution history
 - Error recovery history
-

8. Database Integration

Results can be automatically uploaded to:

- Pan-M Database

- Internal laboratory databases
 - Cloud storage systems
 - Research portals
-

Technical Architecture

User ↓ Antigravity Agent ↓ Workflow Planner ↓ Command Executor ↓ Log Monitor ↓ Error Analyzer ↓ Auto-Fix Engine ↓ Results Manager ↓ Database Integration

MVP Scope

The initial prototype will focus on proving autonomous execution and recovery.

MVP Features

- Accept FASTQ files or SRA IDs
 - Run BWA-MEM2
 - Run samtools
 - Run bcftools
 - Monitor execution logs
 - Detect failures
 - Apply automated fixes
 - Retry failed steps
 - Generate final report
-

Development Roadmap

Phase 1

Manual Pipeline Execution

Goals:

- Install tools
- Run small datasets
- Validate workflow

Deliverable:

Working alignment and variant-calling pipeline

Phase 2

Agent Execution Layer

Goals:

- Python subprocess execution
- Log collection
- Progress monitoring

Deliverable:

Agent-controlled pipeline execution

Phase 3

Failure Detection

Goals:

- Error classification
- Failure logging
- Recovery recommendations

Deliverable:

Automated error diagnosis

Phase 4

Self-Healing Workflows

Goals:

- Automatic fixes
- Retry mechanisms
- Workflow continuation

Deliverable:

Autonomous pipeline recovery

Phase 5

Database Integration

Goals:

- Webhook support
- VCF uploads
- Metadata storage

Deliverable:

End-to-end automated workflow

Success Criteria

The project is considered successful when:

A user submits a genomic dataset and receives a completed analysis without manually debugging pipeline failures.

Example:

Input:

Analyze SRR12345678

Output:

- ✓ Reads downloaded
- ✓ Alignment completed
- ✓ Variants identified
- ✓ One execution error detected
- ✓ Error corrected automatically
- ✓ Pipeline resumed successfully
- ✓ Results uploaded

✓ Final report generated

Long-Term Vision

Build a fully autonomous genomic operations platform that enables researchers to focus on biological insights rather than computational troubleshooting.

The Autonomous Bioinformatics Agent will function as an AI-powered bioinformatics engineer capable of planning, executing, monitoring, repairing, and reporting on genomic analyses with minimal human intervention.